

Практическая работа № 3. Работа с последовательностями в Python

Цель работы

Изучить основы работы с массивами и последовательностями различных типов: списки, множества, кортежи и словари. Понять разницу между изменяемыми и неизменяемыми последовательностями, а также освоить основные операции над ними.

Задания

1. Списки.

- Создайте список из 5 элементов, содержащий числа от 1 до 5.
- Добавьте в конец списка число 6.
- Удалите второй элемент списка.
- Измените третий элемент списка на 10.
- Выведите список на экран.
- Создайте новый список, содержащий только четные числа из исходного списка.
- Отсортируйте список по убыванию.
- Найдите сумму всех элементов списка.

```
my_list = [1, 2, 3, 4, 5]
my_list._____(6)
____ my_list[1]
my_list[_] = ____
print(my_list)
```

```
even_numbers = [x for x in my_list if x ____]
sorted_list = _____
sum_of_elements = _____
```

```
print(even_numbers)
print(sorted_list)
print(sum_of_elements)
```

Задание 1.2

- Создайте список из 100 случайных чисел от 1 до 1000.
- Разделите числа на две группы: четные и нечетные.
- Найдите среднее значение для каждой группы.

- Выведите топ-5 наибольших чисел из каждой группы.

Для задания понадобится использовать модуль **random** и метод **randint**:

```
import random
random.randint(1, 1000)
```

Для выделения топ-5 используйте метод **sorted** и **слайсы**.

Оформите вывод в консоль:

```
print("Среднее значение четных:", _____)
print("Среднее значение нечетных:", _____)
print("Топ-5 четных, _____)
print("Топ-5 нечетных:", _____)
```

2. Кортежи

- Создайте кортеж из 3 элементов, содержащий строки: "apple", "banana", "cherry".
- Попробуйте изменить второй элемент кортежа на "orange". Объясните, почему это невозможно.
- Выведите кортеж на экран.
- Создайте новый кортеж, объединив исходный кортеж с кортежем ("orange", "grape").
- Найдите индекс элемента "banana" в кортеже.
- Проверьте, содержится ли "apple" в кортеже.

```
my_tuple = ("apple", "banana", "cherry")
my_tuple[1] = "orange" # Это вызовет ошибку, так как
кортежи неизменяемы
print(my_tuple)
```

```
new_tuple = _____
index_of_banana = _____
contains_apple = _____
```

```
print(new_tuple)
print(index_of_banana)
print(contains_apple)
```

3. Множества.

- Создайте множество из элементов: 1, 2, 3, 4, 5.
- Добавьте в множество число 6.
- Удалите число 3 из множества.
- Проверьте, содержится ли число 4 в множестве.
- Выведите множество на экран.
- Создайте второе множество из элементов: 4, 5, 6, 7, 8.
- Найдите пересечение двух множеств.
- Найдите объединение двух множеств.
- Найдите числа, которые есть в обоих списках, но не в их пересечении.

```
my_set = {1, 2, 3, 4, 5}
my_set.__(6)
my_set.__(3)
print(4 _____)
print(my_set)
```

```
second_set = _____
intersection = my_set._____(second_set)
union = my_set._____(second_set)
xor_union = _____
print(intersection)
print(union)
print(xor_union)
```

4. Словари

- Создайте словарь, где ключами будут имена студентов, а значениями — их оценки (например: {"Alice": 85, "Bob": 90}).
- Добавьте несколько студентов в словарь.
- Измените оценку одного из студентов.
- Удалите одного студента из словаря.
- Выведите словарь на экран.
- Найдите студента с самой высокой оценкой.
- Найдите средний балл студентов.
- Выведите имена студентов с баллом ниже среднего в консоль.

- Создайте новый словарь, где ключи и значения поменяются местами.
- Проверьте, содержится ли студент "Alice" в словаре.

```
grades = {"Alice": 85, "Bob": 90}

_____
_____
_____
print(grades)

top_student = max(_____)
avg_student = _____
reversed_grades = {_____ for k, v in _____}
contains_alice = _____

print(top_student)
print("avg", avg_student)
print("Студенты с баллом ниже среднего:")
for _____ in grades_____:
    if _____:
        print(_____)
print(reversed_grades)
print(contains_alice)
```

5. Сравнение изменяемых и неизменяемых последовательностей

- Создайте список и кортеж с одинаковыми элементами.
- Попробуйте изменить элемент в списке и кортеже. Объясните разницу.
- Выведите оба объекта на экран.
- Создайте новый список, содержащий элементы кортежа.
- Создайте новый кортеж, содержащий элементы списка.
- Проверьте, можно ли добавить элемент в кортеж.

```
my_list = _____
my_tuple = _____

my_list[0] = 10 # Это сработает, так как списки изменяемы
my_tuple[0] = 10 # Это вызовет ошибку, так как кортежи
неизменяемы
```


new_list_from_tuple = _____
new_tuple_from_list = _____

new_tuple_from_list.append(4) # Что произойдет если
выполнить эту строку? Почему?

6. Операции над последовательностями

- Создайте два списка: [1, 2, 3] и [4, 5, 6].
- Объедините их в один список.
- Создайте множество из объединенного списка.
- Преобразуйте множество обратно в список и отсортируйте его по убыванию.
- Выведите результат на экран.
- Создайте список, содержащий только уникальные элементы из объединенного списка.
- Найдите минимальный и максимальный элементы в списке.
- Подсчитайте количество элементов в списке.

```
list1 = [1, 2, 3]
list2 = [4, 5, 6]
combined_list = _____
combined_set = set(_____)
sorted_list = sorted(_____)
_____
```

```
unique_list = list(_____)
min_element = _____
max_element = _____
count_elements = _____
```

```
print(_____)
print(_____)
print(_____)
```

```
print(_____)
```

7. Индексы и срезы.

- Создайте строку с помощью функции **input**.
- Найдите самое длинное слово в тексте и переверните его.
- Выведите все слова длиной не менее 5 символов.

8. Модуль collections.

- Создайте строку.
- Используйте Counter для поиска самых частых букв в тексте.
- Используйте deque для моделирования очереди из 5 человек.

```
from collections import Counter, deque
```

```
text = "абракадабра"  
char_count = Counter(_____)
```

```
print("Самые частые буквы:", char_count._____ (3))
```

```
queue = deque(["Иван", "Мария", "Петр", "Ольга",  
"Антон"])  
queue._____ # Новый человек встает в очередь  
queue._____ # Первый уходит
```

```
print("Очередь после изменений:", queue)
```

Задание 8.1

- Создайте OrderedDict, хранящий порядок сдачи экзаменов студентами.
- Выведите их в порядке регистрации.
- Добавьте нового студента и убедитесь, что порядок сохраняется.
- Переместите одного студента в начало очереди используя метод **move_to_end**.

```
from collections import OrderedDict
```

```
exam_order = OrderedDict()  
exam_order["___"] = "Сдал"  
...
```

```
print("Порядок сдачи экзаменов:", _____)
```

```
exam_order["_____"] = "_____"
print("После добавления нового студента:", _____)

exam_order._____("_____", _____)
print("После перемещения _____ в начало:", _____)
```

Задание 8.2

- Создайте namedtuple для представления студента с полями:
 - Имя
 - Группа
 - Средний балл
- Создайте список студентов.
- Выведите информацию о каждом студенте.
- Выведите информацию о студенте с максимальным средним баллом.

```
from collections import namedtuple

Student = namedtuple("Student", [_____])

_____ = [
    Student("Иван", "БИ-21", 4.5),
    ...
]

avg = 0
for student in _____:
    print(student)
    if student._____ > avg:
        avg = student._____
        top = student

print("Студент с самым высоким баллом:", top)
```

9. Допишите в файл программы для обработки формул в формате SMILES дополнительный код, который будет описывать список разделителей и список элементов с их атомной массой. Добавьте к этому возможность по введенному в командную строку обозначению элемента выводить его атомную массу в консоль.

```
delimiters = ["[", "]", "=", "-", "#", "/", "\\", "(", ")", " "]

[( 'Al', 13, 26.982 ) ( 'Sb', 51, 121.76 ) ( 'Ar', 18,
39.948 )
( 'As', 33, 74.922 ) ( 'Ba', 56, 137.33 ) ( 'Be', 4,
9.0122)
( 'Bi', 83, 208.98 ) ( 'B', 5, 10.806 ) ( 'Br', 35,
79.901 )
( 'Cd', 48, 112.41 ) ( 'Cs', 55, 132.91 ) ( 'Ca', 20,
40.078 )
( 'C', 6, 12.009 ) ( 'Ce', 58, 140.12 ) ( 'Cl', 17,
35.446 )]
```

***Данный файл в дальнейшем будет дорабатываться. ОБЯЗАТЕЛЬНО СОХРАНИТЕ ЕГО! В результате доработок окончательный проект будет содержать обработку химической формулы в формате SMILES и первичный анализ соединения.**

Вопросы для самоконтроля

- В чем разница между списками и кортежами?
- Какие операции можно выполнять над множествами, но нельзя над списками?
- Почему словари считаются изменяемыми объектами?
- Какие методы можно использовать для работы со списками?
- Как можно преобразовать список в множество и наоборот?
- Какие операции доступны для работы с кортежами?